Partie IA : détecter une attaque automatiquement

PROJECT RESILIENCE-ALPHA · Workshop EPSI Bac+5 · Équipe G[n] · Auteur : [prénom, nom] · Point d'avancement du lundi

1. Pourquoi une partie IA ?

Le sujet demande une plateforme de paiement capable de survivre à une attaque et de se rétablir sans qu'un opérateur intervienne. Pour qu'une réponse automatique se déclenche, il faut d'abord que le système **s'aperçoive tout seul** que quelque chose ne va pas. C'est le rôle de la partie IA.

Le sujet précise que cette détection ne doit pas se réduire à quelques conditions codées en dur du type « si le processeur dépasse 90 %, alerter ». Nous avons donc choisi un modèle de machine learning qui apprend lui-même ce qui est normal.

La partie IA se compose de deux éléments distincts :

- un **détecteur d'anomalies**, qui surveille les métriques de la plateforme et produit un score ;
- un **assistant pour l'opérateur**, qui l'aide à comprendre l'incident en consultant nos procédures.

Ces deux éléments sont volontairement séparés. Le détecteur ne répare rien : il donne un score, et c'est l'orchestrateur de l'équipe qui décide, puis Ansible qui exécute. L'assistant, lui, ne fait que lire des documents et n'a aucun accès au système. Ce choix limite les risques : une IA qui pourrait agir sur les conteneurs serait une surface d'attaque supplémentaire.

2. Le détecteur d'anomalies

2.1 Ce qu'il mesure

Une **métrique** est un chiffre qui mesure l'état d'un service, relevé régulièrement : c'est le tableau de bord de la plateforme. Notre détecteur en suit six.

Métrique	Ce qu'elle mesure
Requêtes par seconde	Le volume de demandes reçues par l'API
Erreurs 5xx par seconde	Les fois où l'API plante
Erreurs 4xx par seconde	Les demandes refusées (mauvais mot de passe, par exemple)
Latence p95	Le temps de réponse que 95 % des requêtes ne dépassent pas
CPU	La part de processeur utilisée par l'API
Blocages du WAF	Les requêtes arrêtées par le pare-feu applicatif

Prises ensemble, ces six valeurs forment une « photo » de la plateforme à un instant donné. En fonctionnement normal, la photo ressemble à peu près à ceci : une vingtaine de requêtes par seconde, presque aucune erreur, une réponse en 0,08 seconde, un processeur à 30 %. Pendant une saturation de l'API, elle devient : environ 300 requêtes par seconde, des erreurs serveur, une réponse en plus de 2 secondes et un processeur proche de 100 %.

2.2 Comment il apprend

Nous utilisons un algorithme appelé **Isolation Forest**. Son principe est simple : dans un ensemble de mesures, les situations anormales sont rares et différentes des autres, donc plus faciles à « isoler ». L'algorithme construit de nombreux arbres de décision aléatoires et regarde combien de questions il faut poser pour isoler chaque mesure. Une mesure ordinaire demande beaucoup de questions. Une mesure inhabituelle en demande peu, et reçoit un score d'anomalie élevé.

Nous l'avons choisi pour trois raisons :

1. Il est **non supervisé** : il apprend uniquement à partir de trafic normal et n'a pas besoin d'exemples d'attaques étiquetés, que nous n'aurions pas au départ.
2. Il est **rapide**, donc compatible avec une détection toutes les quelques secondes sur un PC de travail.
3. Il est **explicable** : on peut dire pourquoi une mesure est jugée anormale (ses valeurs s'éloignent de ce que le modèle a vu).

Avant l'entraînement, les six variables sont **normalisées**, c'est-à-dire ramenées à la même échelle. Sans cela, le nombre de requêtes (des centaines) écraserait la latence (des fractions de seconde).

2.3 Comment il s'intègre à la chaîne de reprise

Métriques de la plateforme > Détecteur (score) > Alerte > Orchestrateur > Action Ansible > Vérification

Le détecteur est un petit service web. Toutes les quelques secondes, il lit les mesures, calcule un **score d'anomalie** et un **indicateur** (0 ou 1), puis les publie sur une page /metrics que Prometheus, l'outil de supervision de l'équipe, vient lire. Une alerte se déclenche lorsque l'indicateur reste à 1 pendant 15 secondes : cette durée évite qu'un pic isolé provoque une fausse alerte. L'orchestrateur reçoit ensuite cette alerte et décide de l'action à mener.

3. L'assistant pour l'opérateur

Pendant un incident, un opérateur peut avoir besoin de réponses rapides : que signifie ce score, quelle est la procédure de restauration, où voir les journaux. L'assistant répond à ces questions à partir des documents de l'équipe (runbooks, description de l'architecture, procédures).

Il repose sur trois briques complémentaires, qui ne sont pas des alternatives :

- **Ollama** fait tourner un modèle de langage en local, sans service externe ;
- le **RAG** (recherche documentaire enrichissant un modèle) consiste à chercher d'abord les passages utiles dans nos documents, puis à demander au modèle de répondre en s'appuyant sur eux, ce qui l'empêche d'inventer des procédures ;
- **LangChain** relie ces étapes : chargement des documents, recherche, appel du modèle.

L'assistant est en **lecture seule**. Il n'a accès ni à Docker, ni à Ansible, ni à la base de données, ni aux secrets. Il explique, il ne répare pas.

4. Ce qui est réalisé aujourd'hui

Le détecteur est fonctionnel **sur des données de test**. En l'absence de plateforme en marche ce lundi, nous avons fabriqué un jeu de données synthétique : 600 mesures de trafic normal et 60 mesures d'attaque simulée. Ces données nous permettent de développer et de tester tout le code en attendant les vraies métriques.

Sur ce jeu de test, le modèle juge anormales environ **1 % des mesures de trafic normal** et **100 % des mesures d'attaque**. Un service FastAPI qui publie le score est écrit et en cours de test en local.

Ces chiffres doivent être lus avec prudence. L'attaque que nous avons inventée est très éloignée du trafic normal, donc la détecter est facile, et le test sur le trafic normal utilise les données d'entraînement. Ils montrent que la mécanique fonctionne. Ils ne prouvent pas encore la qualité de la détection sur la vraie plateforme.

Élément	État
Environnement de travail, modèle entraîné, évaluation sur données de test	Fait
Service détecteur qui publie le score	Écrit, tests locaux en cours
Entraînement sur des métriques réelles	À faire (mardi)
Assistant (documents, Ollama, LangChain)	À faire
Mesure du délai de détection sur la vraie attaque	À faire (mercredi)

5. Ce qu'il nous faut de l'équipe

Pour remplacer les données de test par des données réelles, nous avons besoin que la plateforme tourne et expose ses métriques. Concrètement :

- que l'API de paiement expose une page /metrics avec le nombre de requêtes par statut et par route, et la durée des requêtes ;
- que Prometheus collecte ces métriques, et que son adresse nous soit communiquée ;
- que l'orchestrateur sache lire notre score ;
- qu'une attaque fonctionnelle soit disponible, pour mesurer en combien de secondes nous la détectons.

Le plan est ensuite simple : nous générons un trafic normal varié pendant plusieurs dizaines de minutes, nous collectons les métriques correspondantes, nous réentraînons le modèle sur ces données, puis nous lançons l'attaque pour vérifier que le score monte.

6. Limites

- **Données actuelles inventées.** Tant qu'elles ne sont pas remplacées par des métriques réelles, les résultats restent indicatifs.
- **Fausses alertes possibles.** Une montée de charge légitime peut ressembler à une attaque. Nous la limitons en entraînant le modèle sur des charges variées et en exigeant 15 secondes d'anomalie continue avant d'alerter.

- **Le modèle ne donne pas le type d'attaque.** Il dit que quelque chose est anormal, pas quoi. La recherche de la source (quel compte, quelle adresse) se fait par l'orchestrateur, avec les journaux.
- **Une attaque discrète peut passer inaperçue** si elle ne modifie pas les six métriques suivies.

7. Prochaines étapes

Jour	Objectif
Mardi	Collecte des métriques réelles, réentraînement, connexion du détecteur à Prometheus
Mercredi	Assistant opérateur branché au tableau de bord, intégration avec l'orchestrateur, mesure du délai de détection et des fausses alertes
Jeudi	Résultats finaux dans le dossier, répétitions de la démonstration